Building REST API Using Spring Boot

Building a RESTful API is a common task for backend developers. With Spring Boot, creating robust and scalable REST APIs becomes straightforward. This guide walks you through the process of building a REST API using Spring Boot, covering essential steps, best practices, and useful tips.

What is Spring Boot?

Spring Boot is an extension of the Spring framework that simplifies the development of stand-alone, production-ready Spring applications. It provides pre-configured settings, reducing boilerplate code and allowing developers to focus on business logic.

Prerequisites

Before you start, ensure you have the following installed:
- Java Development Kit (JDK) 8 or higher
- Maven or Gradle
- An Integrated Development Environment (IDE) like IntelliJ IDEA or Eclipse

Step 1: Setting Up Your Project

Using Spring Initializr

1. Visit [Spring Initializr](https://start.spring.io/).
2. Configure your project:
— Project: Maven Project
— Language: Java
— Spring Boot: (select the latest stable version)
— Group: com.example
— Artifact: rest-api
— Name: rest-api
— Description: Demo project for Spring Boot REST API
— Package Name: com.example.restapi
— Packaging: Jar
— Java: 8 or higher

3. Add dependencies:
— Spring Web
— Spring Data JPA
— H2 Database (for in-memory database)

4. Click on “Generate” to download the project zip file.
5. Extract the zip file and open it in your IDE.

Step 2: Creating the Data Model

Create a simple data model class that represents the data your API will handle. In this example, we’ll create a `Book` entity.

Book.java

```
package com.example.restapi.model;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
```

```
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;
import java.util.Objects;

@Entity
@Table(name = "books") // Optional: Specify table name explicitly
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String author;
    private String isbn;

    // No-argument constructor
    public Book() {
    }

    // All-argument constructor (optional, for convenience)
    public Book(String title, String author, String isbn) {
        this.title = title;
        this.author = author;
        this.isbn = isbn;
    }

    // Getters and setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }
```

```
    public void setAuthor(String author) {
        this.author = author;
    }

    public String getIsbn() {
        return i
```

Step 3: Creating the Repository

Create a repository interface that extends `JpaRepository` to handle data access.

BookRepository.java

```
package com.example.restapi.repository;
import com.example.restapi.model.Book;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface BookRepository extends JpaRepository<Book, Long> {
}
```

Step 4: Creating the Service Layer

Create a service class that encapsulates the business logic and interacts with the repository.

BookService.java

```
package com.example.restapi.service;

import com.example.restapi.model.Book;
import com.example.restapi.repository.BookRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class BookService {

    private final BookRepository bookRepository;

    @Autowired
    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public List<Book> getAllBooks() {
        return bookRepository.findAll();
```

```
    }

    public Optional<Book> getBookById(Long id) {
        return bookRepository.findById(id);
    }

    public Book createBook(Book book) {
        return bookRepository.save(book);
    }

    public void deleteBook(Long id) {
        if (bookRepository.existsById(id)) {
            bookRepository.deleteById(id);
        } else {
            throw new RuntimeException("Book not found with id: " + id);
        }
    }
}
```

Step 5: Creating the Controller

Create a REST controller to handle HTTP requests and map them to the service methods.

BookController.java

```
package com.example.restapi.controller;

import com.example.restapi.model.Book;
import com.example.restapi.service.BookService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/books")
public class BookController {

    @Autowired
    private BookService bookService;

    @GetMapping
    public List<Book> getAllBooks() {
        return bookService.getAllBooks();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Book> getBookById(@PathVariable Long id) {
```

```
        Optional<Book> book = bookService.getBookById(id);
        return book.map(ResponseEntity::ok).orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PostMapping
    public Book createBook(@RequestBody Book book) {
        return bookService.createBook(book);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteBook(@PathVariable Long id) {
        bookService.deleteBook(id);
        return ResponseEntity.noContent().build();
    }
}
```

Step 6: Configuring the Application Properties

Configure the application properties for the in-memory H2 database.

application.properties

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=root
spring.datasource.password=password
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
```

Step 7: Running the Application

Run the application using your IDE or by executing the following command in the terminal:

```
mvn spring-boot:run
```

Step 8: Testing the API

Use a tool like Postman or cURL to test your API endpoints.

Examples:

Get all books:

GET http://localhost:8080/api/books

Get a book by ID:

GET http://localhost:8080/api/books/1

Create a new book:

POST http://localhost:8080/api/books
Content-Type: application/json

```
{
 "title": "Spring Boot in Action",
 "author": "Craig Walls",
 "isbn": "9781617292545"
}
```

Delete a book:

DELETE http://localhost:8080/api/books/1

Let's break down the flow of the Spring Boot REST API from the moment a request is sent to when a response is generated.

1. Client Sends a Request

When a client (like a browser, Postman, or another service) sends an HTTP request to your API, it targets a specific endpoint. For example, a GET request to `http://localhost:8080/api/books`.

2. Request Mapping to Controller

The Spring framework intercepts the incoming request and maps it to the appropriate controller method based on the URL and HTTP method.

Example:
For a GET request to `/api/books`, Spring looks for a method in a controller class annotated with `@RestController` and mapped to `/api/books`.

3. Controller Method Execution

The `BookController` class has methods annotated with `@GetMapping`, `@PostMapping`, `@DeleteMapping`, etc., which correspond to different HTTP methods.

Example:

```
package com.example.restapi.controller;

import com.example.restapi.model.Book;
import com.example.restapi.service.BookService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/books")
public class BookController {

    @Autowired
    private BookService bookService;

    @GetMapping
```

```
    public List<Book> getAllBooks() {
        // Fetches all books from the service layer and returns them
        return bookService.getAllBooks();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Book> getBookById(@PathVariable Long id) {
        // Fetches a book by its ID from the service layer
        Optional<Book> book = bookService.getBookById(id);
        // Returns the book if found; otherwise, returns a 404 Not Found response
        return book.map(ResponseEntity::ok).orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PostMapping
    public Book createBook(@RequestBody Book book) {
        // Creates a new book using the service layer and returns the created book
        return bookService.createBook(book);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteBook(@PathVariable Long id) {
        // Deletes the book with the specified ID using the service layer
        bookService.deleteBook(id);
        // Returns a 204 No Content response to indicate successful deletion
        return ResponseEntity.noContent().build();
    }
}
```

4. Service Layer Invocation

The controller methods typically call methods in a service layer to perform business logic. The `BookService` class handles the interaction with the database through the `BookRepository`.

Example:

```
@Service
public class BookService {
    private final BookRepository bookRepository;

    @Autowired
    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }

    public Optional<Book> getBookById(Long id) {
        return bookRepository.findById(id);
```

```
    }

    public Book createBook(Book book) {
        return bookRepository.save(book);
    }

    public void deleteBook(Long id) {
        bookRepository.deleteById(id);
    }
}
```

5. Repository Interaction

The service layer uses a repository interface that extends `JpaRepository`. This repository handles CRUD operations on the database. The repository methods are automatically implemented by Spring Data JPA.

Example:

```
@Repository
public interface BookRepository extends JpaRepository<Book, Long> {
}
```

6. Data Model and Database Interaction

The `Book` class is an entity that maps to a database table. Spring Data JPA uses this mapping to perform operations on the corresponding table.

Example:

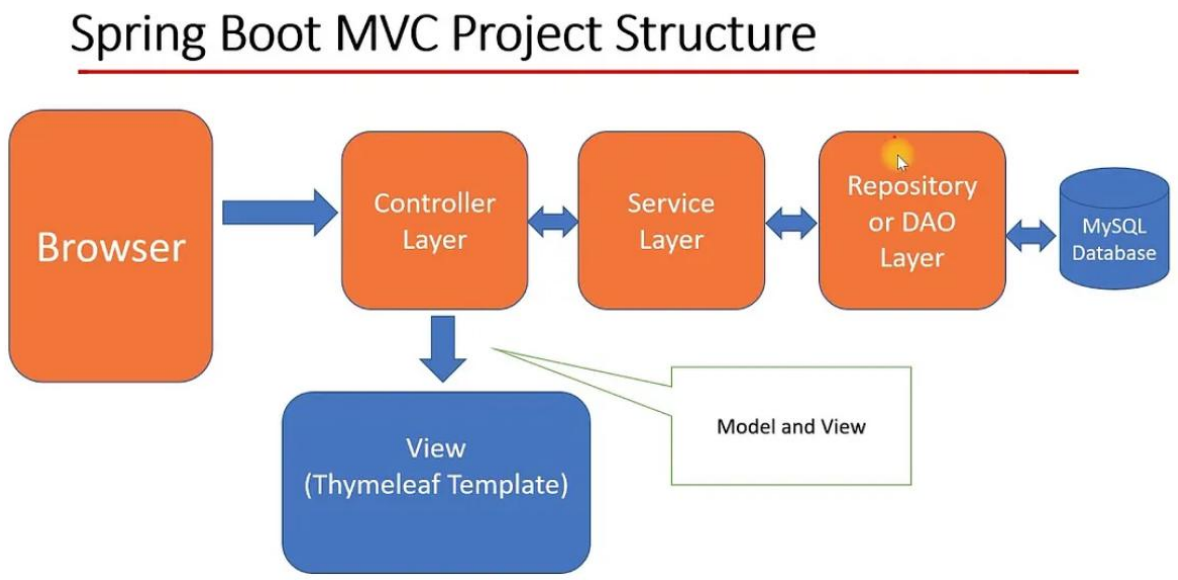
```
 @Entity
 public class Book {

     @Id
     @GeneratedValue(strategy = GenerationType.IDENTITY)
     private Long id;
     private String title;
     private String author;
     private String isbn;
    // Getters and setters

}
```

7. Response Generation

After the service layer completes its operations (e.g., fetching data from the database, saving a new entity), the data is returned to the controller. The controller then wraps this data into an appropriate response format (e.g., JSON) and returns it to the client.

Detailed Flow Example:

Press enter or click to view image in full size



GET Request to `/api/books`

1. Client sends a GET request to `/api/books`.
2. Spring maps the request to `BookController.getAllBooks()`.
3. `getAllBooks()` method in `BookController` is called.
4. `BookController` calls `BookService.getAllBooks()`.
5. `BookService` calls `BookRepository.findAll()`.
6. Spring Data JPA performs a SELECT query on the `books` table.
7. Database returns the list of books to `BookRepository`.
8. `BookRepository` returns the list to `BookService`.
9. `BookService` returns the list to `BookController`.
10. `BookController` returns the list as a JSON response to the client.

POST Request to `/api/books`

1. Client sends a POST request to `/api/books` with a JSON payload representing a new book.
2. Spring maps the request to `BookController.createBook()`.
3. `createBook()` method in `BookController` is called with the book data from the request body.
4. `BookController` calls `BookService.createBook()`.
5. `BookService` calls `BookRepository.save(book)`.
6. Spring Data JPA performs an INSERT query on the `books` table.
7. Database saves the new book and returns the saved entity with an ID.
8. `BookRepository` returns the saved book to `BookService`.
9. `BookService` returns the saved book to `BookController`.
10. `BookController` returns the saved book as a JSON response to the client.

Summary

- Controller: Handles HTTP requests and sends responses.
- Service: Contains business logic and interacts with repositories.
- Repository: Manages data access and CRUD operations using Spring Data JPA.
- Entity: Represents the data model and maps to a database table.

This flow ensures a clean separation of concerns, making the codebase easier to maintain and test. By following these steps, Spring Boot efficiently processes requests and generates responses, enabling the creation of scalable and robust REST APIs.

Data Transfer Objects (DTOs)

When building a REST API in Java using Spring Boot, it's common to use Data Transfer Objects (DTOs) to encapsulate data that will be sent or received by the API. DTOs are simple objects that only contain fields and getter/setter methods, without any business logic, and are used to transfer data between layers of the application.

What is a DTO?

Think of a Data Transfer Object (DTO) as a special package for carrying data from one place to another. Imagine you're sending a gift to a friend; you wouldn't send the entire store — just the gift you want them to have. Similarly, a DTO is a way to send just the specific pieces of information that are needed.

Why Use DTOs?

1. Keep Things Simple:
— Without DTO: If you send your whole "book" (which might have lots of extra details), it can be too much for the recipient.
— With DTO: You create a simple "book package" that includes only the information needed, like just the title and author, not the entire book.

2. Protect Sensitive Information:
— Without DTO: You might accidentally send information that you don't want others to see.
— With DTO: You can make sure to only include safe and necessary information, like omitting sensitive details such as personal notes or internal IDs.

3. Improve Performance:
— Without DTO: Sending a lot of data can slow things down, like a slow-loading web page.
— With DTO: You send a smaller, focused amount of data, which can make things faster and more efficient.

4. Flexibility:
— Without DTO: If you need to change how your data is stored or represented, it can impact everything that uses it.
— With DTO: You can change your internal data setup without affecting how the data is sent or received, which makes it easier to update and maintain your system.

5. Organize Data:
— Without DTO: Data from different sources might be mixed together in a confusing way.
— With DTO: You can neatly package and format data so it's easier for others to understand and use.

Example in Simple Terms

Imagine you're organizing a party and you want to send invitations. Instead of sending the entire guest list with every detail about each guest, you just send a simple invitation with the name and RSVP status.

Without DTO: You might send the whole guest list with all their contact details, addresses, and even some personal notes — stuff that's not needed for the invitation.

With DTO: You send just the essential details needed for the invite — like the guest's name and RSVP status — making it clear and focused.

In a software application, a DTO does something similar. It helps you send only the data that's necessary for the task at hand, making things simpler and more efficient.

Here's a detailed example of a REST API that uses DTOs. The example includes creating, retrieving, updating, and deleting a `Product` resource.

Project Structure

```
src
 ├── main
|   ├── java
|   |   ├── com
|   |   |   ├── example
|   |   |   |   ├── demo
|   |   |   |   |   ├── controller
|   |   |   |   |   |   ├── ProductController.java
|   |   |   |   |   ├── dto
|   |   |   |   |   |   ├── ProductDTO.java
|   |   |   |   |   ├── entity
|   |   |   |   |   |   ├── Product.java
|   |   |   |   |   ├── service
|   |   |   |   |   |   ├── ProductService.java
|   |   |   |   |   ├── repository
|   |   |   |   |   |   ├── ProductRepository.java
|   |   |   |   |   ├── DemoApplication.java
|   └── resources
|       ├── application.properties
```

1. Entity Class

The `Product` entity represents the data stored in the database.

```
package com.example.demo.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
```

```
    private String description;
    private double price;

    // Constructors, getters, and setters

    public Product() {}

    public Product(String name, String description, double price) {
        this.name = name;
        this.description = description;
        this.price = price;
    }

    // Getters and Setters
}
```

2. DTO Class

The `ProductDTO` is a simple class used to transfer data. It contains only the fields you want to expose in your API.

```
package com.example.demo.dto;

public class ProductDTO {

    private Long id;
    private String name;
    private String description;
    private double price;

    // Constructors, getters, and setters

    public ProductDTO() {}

    public ProductDTO(Long id, String name, String description, double price) {
        this.id = id;
        this.name = name;
        this.description = description;
        this.price = price;
    }

    // Getters and Setters
}
```

3. Repository Interface

The `ProductRepository` is an interface for performing database operations.

```
package com.example.demo.repository;
```

```
import com.example.demo.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProductRepository extends JpaRepository<Product, Long> {
}
```

4. Service Class

The `ProductService` class contains the business logic of the application. It uses the repository to perform CRUD operations and handles the conversion between `Product` and `ProductDTO`.

```
package com.example.demo.service;

import com.example.demo.dto.ProductDTO;
import com.example.demo.entity.Product;
import com.example.demo.repository.ProductRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class ProductService {

    @Autowired
    private ProductRepository productRepository;

    // Convert Product entity to ProductDTO
    private ProductDTO mapToDTO(Product product) {
        return new ProductDTO(product.getId(), product.getName(), product.getDescription(),
product.getPrice());
    }

    // Convert ProductDTO to Product entity
    private Product mapToEntity(ProductDTO productDTO) {
        return new Product(productDTO.getName(), productDTO.getDescription(),
productDTO.getPrice());
    }

    // Get all products
    public List<ProductDTO> getAllProducts() {
        return productRepository.findAll().stream()
                .map(this::mapToDTO)
                .collect(Collectors.toList());
    }

    // Get a single product by ID
    public ProductDTO getProductById(Long id) {
```

```
        Product product = productRepository.findById(id).orElseThrow(() -> new
RuntimeException("Product not found"));
        return mapToDTO(product);
    }

    // Create a new product
    public ProductDTO createProduct(ProductDTO productDTO) {
        Product product = mapToEntity(productDTO);
        Product savedProduct = productRepository.save(product);
        return mapToDTO(savedProduct);
    }

    // Update an existing product
    public ProductDTO updateProduct(Long id, ProductDTO productDTO) {
        Product existingProduct = productRepository.findById(id).orElseThrow(() -> new
RuntimeException("Product not found"));
        existingProduct.setName(productDTO.getName());
        existingProduct.setDescription(productDTO.getDescription());
        existingProduct.setPrice(productDTO.getPrice());
        Product updatedProduct = productRepository.save(existingProduct);
        return mapToDTO(updatedProduct);
    }

    // Delete a product
    public void deleteProduct(Long id) {
        Product product = productRepository.findById(id).orElseThrow(() -> new
RuntimeException("Product not found"));
        productRepository.delete(product);
    }
}
```

5. Controller Class

The `ProductController` class exposes REST endpoints and uses `ProductService` to handle requests.

```
package com.example.demo.controller;

import com.example.demo.dto.ProductDTO;
import com.example.demo.service.ProductService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/products")
public class ProductController {
```

```
    @Autowired
    private ProductService productService;

    // Get all products
    @GetMapping
    public ResponseEntity<List<ProductDTO>> getAllProducts() {
        List<ProductDTO> products = productService.getAllProducts();
        return ResponseEntity.ok(products);
    }

    // Get a product by ID
    @GetMapping("/{id}")
    public ResponseEntity<ProductDTO> getProductById(@PathVariable Long id) {
        ProductDTO product = productService.getProductById(id);
        return ResponseEntity.ok(product);
    }

    // Create a new product
    @PostMapping
    public ResponseEntity<ProductDTO> createProduct(@RequestBody ProductDTO productDTO) {
        ProductDTO createdProduct = productService.createProduct(productDTO);
        return ResponseEntity.ok(createdProduct);
    }

    // Update a product
    @PutMapping("/{id}")
    public ResponseEntity<ProductDTO> updateProduct(@PathVariable Long id, @RequestBody
ProductDTO productDTO) {
        ProductDTO updatedProduct = productService.updateProduct(id, productDTO);
        return ResponseEntity.ok(updatedProduct);
    }

    // Delete a product
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }
}
```

6. Example of API Endpoints

- GET `/api/products` — Get all products
- GET `/api/products/{id}` — Get a product by ID
- POST `/api/products` — Create a new product
- PUT `/api/products/{id}` — Update an existing product
- DELETE `/api/products/{id}` — Delete a product

7. Testing with Postman

You can test these endpoints using Postman:

1. Create a Product:
   — URL: `POST /api/products`
   — Body:

```
{
  "name": "Laptop",
  "description": "High-end gaming laptop",
  "price": 1500.00
}
```

2. Get All Products:
— URL: `GET /api/products`

3. Get Product by ID:
— URL: `GET /api/products/{id}`

4. Update a Product:
— URL: `PUT /api/products/{id}`
— Body:

```
{
  "name": "Updated Laptop",
  "description": "Updated description",
  "price": 1600.00
}
```

5. Delete a Product:
— URL: `DELETE /api/products/{id}`

This example shows how to use DTOs in a Spring Boot REST API. The DTOs help decouple the internal representation of the `Product` entity from the API's external representation. The `ProductService` handles the business logic, and the `ProductController` exposes the API endpoints. This pattern is useful in large applications where separating concerns and controlling the data exposure is important.